1. Limitless
1.1. #Template
1.1.1. Python Template
1.2. Data Structures
1.2.1. 2D RMQ
1.2.2. Persistent Treap
1.2.3. RB Tree
1.2.4. Rectangle Union O(n log n)
1.3. Dynamic Programing
1.3.1. Convex Hull Optimization
1.3.2. from DCopt to knuth opt
1.4. Geometry
1.4.1. Delaunay Triangulation
1.4.2. Pick Theorem O(n)
1.4.3. Primitives
1.5. Graph
1.5.1. Dinic
1.5.2. Dominator Tree O((N+M)logN)
1.5.3. DSU on Tree
1.5.4. flow_with_lower_bound
1.5.5. Hopcroftâ□□Karp Bipartite Matching O(Msqrt(N))
1.5.6. Hungarian O(N^3)
1.5.7. MaxFlowMinCost
1.5.8. Minimum Arborescences O(MlogN)
1.5.9. MO's Algorithm on Tree
1.5.10. Punto de Art. y Bridges O(N)
1.5.11. Stable Marriage
1.5.12. steiner_tree
1.5.13. StoerWagner O(N^3)
1.6. Number Theory
1.6.1. Algoritmo Shanka-Tonelli (x^2 = a(mod p))
1.6.2. Extended GCD (ax+by = gcd(a,b))
1.6.3. Fast Modulo Transform O(NlogN)
1.6.4. fft
1.6.5. Find a primitive root of a prime number
1.6.6. Floyds Cycle-Finding algorithm
1.6.7. Gauss
1.6.8. Inverso de Polinomio
1.6.9. Inverso Modular
1.6.10. Josephus
1.6.11. Linear Recurrence Solver O(N^2logK)
1.6.12. linear_congruences
1.6.13. Mobius
1.6.14. Modular Equations (ax = b(n))
1.6.15. Parametric Self-Dual Simplex method O(n+m)
1.6.16. Partition
1.6.17. Phi
1.6.18. Pholard Rho(sirve)
1.6.19. Shanks' Algorithm O(sqrt(N)) (a^x = b(mod m))
1.6.20. Simpson Rule
1.6.21. SQUARE_FREE.CPP
1.7. String
1.7.1. Aho Corasick
1.7.2. Lyndon Decomposition O(N)
1.7.3. Manacher
1.7.4. Palindrome Tree
1.7.5. suffix automata O(N)
1.7.6. SUFFUX_ARRAY(N log N)
1.7.7. Tandems O(Nlog(N))
1.7.8. Z_AND_PREFIX_FUNCTION

1. Limitless

1.1. #Template

1.1.1. Python Template

```python
import string
import math
import fractions

@memoize
def funcion( s ):
    print s[0:2]
    s = sorted( s )
    return s

arr = []
arr.append( 5 )
arr.append( 1 )
arr = funcion(arr)
print arr[0:5]
for x in range(0, 10):
   arr.append( x )
gg = fractions.gcd(10, 65)

while True:
    try:
        n, c, d  = raw_input().split()
        import math
        print pow( long(c), long(n), long(d) )
    except EOFError:
        break
```

1.2. Data Structures

1.2.1. 2D RMQ

```cpp
void build( ){ // O(n*m*log(n)*log(m))
   for(int i = 0; i < n ; i ++){
     for(int j = 0; j < m ; j ++)
           table[0][0][i][j] = Matrix[i][j];
     for(int lj = 1; lj <= log2( m ); lj ++)
           for(int j = 0; j + (1<<(lj-1)) < m; j ++)
            table[0][lj][i][j] =
                  min(table[0][lj-1][i][j],
                        table[0][lj-1][i][j+(1<<(lj-1))]);
   }
   for(int li = 1; li <= log2(n); li ++ )
     for(int i = 0; i < n; i ++ )
           for(int lj = 0; lj <= log2(m); lj++ )
             for(int j = 0; j < m; j ++ )
                   table[li][lj][i][j] =
                       min(table[li-1][lj][i][j],
                             table[li-1][lj][i+(1<<(li-1))][j]);
```

```
}
int Query(int x1,int y1,int x2,int y2){
   int lenx=x2-x1+1;
    int kx=log2(lenx);
    int leny=y2-y1+1;
    int ky=log2(leny);
   int min_R1 = min ( table[kx][ky][x1][y1] ,
                  table[kx][ky][x1][y2-(1<<ky) + 1] );
   int min_R2 = min ( table[kx][ky][x2-(1<<kx) + 1][y1],
                  table[kx][ky][x2-(1<<kx)+1][y2-(1<<ky)+1] );
   return min ( min_R1, min_R2 );
}
```

1.2.2. Persistent Treap

```
/* Careful with memory and recommended
   to use Garbage Collection  */
typedef struct item* pitem;
struct item {
    int val, sz;
    pitem l, r;
    item ( ) {
        val = 0;
        sz = 1;
        l = r = 0;
    }
};
int sz ( pitem t ) { return (t)? t->sz : 0; }
void upd_sz ( pitem t ) {
    t->sz = sz(t->l) + sz(t->r) + 1;
}
typedef tuple<pitem,pitem> tupla;
tupla split ( pitem v, int k ) {
    if ( !v ) return make_tuple ( v, v );
    pitem l, r, ret;
    ret = new item();
    ret->val = v->val;
    if ( k >= sz(v->l) + 1 ) {
        tie(l,r) = split ( v->r, k-sz(v->l)-1 );
        ret->l = v->l;
        ret->r = l;
        upd_sz ( ret );
        return make_tuple ( ret, r );
    } else {
        tie(l,r) = split ( v->l, k );
        ret->r = v->r;
        ret->l = r;
        upd_sz ( ret );
        return make_tuple( l, ret );
    }
}
pitem merge ( pitem l, pitem r ) {
    if ( !l ) return r;
    if ( !r ) return l;
    pitem clone = new item();
    int tl = sz(l), tr = sz(r);
    if ( rand() % (tl+tr) < tl ) {
        clone->val = l->val;
        clone->l = l->l;
        clone->r = merge ( l->r, r );
    } else {
        clone->val = r->val;
        clone->r = r->r;
        clone->l = merge ( l, r->l );
    }
    upd_sz ( clone );
    return clone;
}
```

1.2.3. RB Tree

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
typedef tree<
int,
null_type,
less<int>,
rb_tree_tag,
tree_order_statistics_node_update>
ordered_set;
ordered_set X; //declaracion
X.insert(1); // insertar
X.erase( X.find( 2 ) ); //eliminar
cout<<*X.find_by_order(1)<<endl;// k-th menor elemento
cout<<X.order_of_key(-5)<<endl;//lower_bound(cant. de menores hay)
```

1.2.4. Rectangle Union O(n log n)

```
struct rectangle {
   ll xl, yl, xh, yh;
};
ll rectangle_area(vector<rectangle> &rs) {
   vector<ll> ys; // coordinate compression
   for (auto r : rs) {
           ys.push_back(r.yl);
           ys.push_back(r.yh);
   }
   sort(ys.begin(), ys.end());
   ys.erase(unique(ys.begin(), ys.end()), ys.end());
   int n = ys.size(); // measure tree
   vector<ll> C(8 * n), A(8 * n);
   function<void(int, int, int, int, int, int)> aux =
                   [&](int a, int b, int c, int l, int r, int k) {
                           if ((a = max(a,l)) >= (b = min(b,r)))
                                   return;
                           if (a == l && b == r) C[k] += c;
                           else {
                                   aux(a, b, c, l, (l+r)/2, 2*k+1);
                                   aux(a, b, c, (l+r)/2, r, 2*k+2);
                           }
                           if (C[k]) A[k] = ys[r] - ys[l];
                           else A[k] = A[2*k+1] + A[2*k+2];
```

```
                    };
    struct event {
            ll x, l, h, c;
    };
    vector<event> es;
    for (auto r : rs) {
            int l = lower_bound(ys.begin(), ys.end(), r.yl)
                                                - ys.begin();
            int h = lower_bound(ys.begin(), ys.end(), r.yh)
                                                - ys.begin();
            es.push_back({ r.xl, l, h, +1 });
            es.push_back({ r.xh, l, h, -1 });
    }
    sort(es.begin(), es.end(), [](event a, event b)
                    {return a.x != b.x ? a.x < b.x : a.c > b.c;});
    ll area = 0, prev = 0;
    for (auto &e : es) {
            area += (e.x - prev) * A[0];
            prev = e.x;
            aux(e.l, e.h, e.c, 0, n, 0);
    }
    return area;
}
```

1.3. Dynamic Programing

1.3.1. Convex Hull Optimization

```
//para buscar maximo
typedef complex<ll> point;
typedef vector<point> hull;
ll cross(point a, point b){return imag(conj(a) * b);}
ll dot(point a, point b){ return real(conj(a) * b); }
void add(point a, hull &ch){
    for(int n = (int)ch.size(); n > 1 &&
        cross(ch[n-1]-ch[n-2], a-ch[n-2]) >= 0; n--)
            ch.pop_back();
    ch.push_back(a);
}
ll eval(point a, hull &ch){
    int lo = 0, hi = (int)ch.size()-1;
    while(lo < hi){
        int m = (lo + hi)/2;
        if( dot(ch[m], a) >= dot(ch[m+1], a) ) hi = m;
        else lo = m + 1;
    }
    return dot(ch[lo], a);
}
hull merge(const hull &a, const hull &b){
    int n =(int)a.size(), m =(int)b.size(), x=0, y=0;
    hull c;
    while(x < n && y < m){
        if(real(a[x]) <= real(b[y])) add(a[x++], c);
        else add(b[y++], c);
    }
    while (x < n) add(a[x++], c);
    while (y < m) add(b[y++], c);
    return c;
}
struct dyn{
    vector<hull> H;
    void add(point p){
        hull h; h.push_back(p);
        for (int i = 0; i < (int)H.size(); ++i){
            hull &ch = H[i];
            if (ch.empty()){ ch = h; return; }
            h = merge(h, ch);
            ch.clear();
        }
        if (!h.empty()) H.push_back(h);
    }
    ll query(point p){
        ll answer = -1ll<<60;
        for (int i = 0; i < (int)H.size(); ++i){
            hull &ch = H[i];
            if(ch.empty()) continue;
            answer = max( answer, eval(p, ch) );
        }
        return answer;
    }
};
```

1.3.2. from DCopt to knuth opt

```
/**dp[i][k] = min( dp[j][k-1] + C[j+1][i] ) conquer and divide
   j < i

dp[k][i] = min( dp[k-1][j-1] + C[j][i] )
   k <= j <= i
*/
for( int i = 1; i <= N; i++ )
dp[1][i] = costo(1,i), O[1][i] = 1;

int opt = 0;
now = 1;
for( int k = 2; k <= K; k++ ){
now ^= 1;
for( int i = N; i >= k; i-- ){
 dp[now][i] = 1LL << 60;
 opt = (i == N)?N:O[now][i+1];
 for( int j = max((int)O[now^1][i],k); j <= opt; j++ )
  if( dp[now][i] > dp[now^1][j-1] + costo(j,i) )
   dp[now][i] = dp[now^1][j-1] + costo(j,i), O[now][i] = j;
 }
}


```

1.4. Geometry

1.4.1. Delaunay Triangulation

```
/*Incremental Randomized Expected O(NlogN)*/
int n; point P[maxn];
struct edge {
```

```
int t, side;
edge ( ) { t = -1, side = 0; }
edge ( int tt, int s ) { t = tt, side = s; }
};
struct triangle {
point p[3]; edge e[3]; int child[3];
triangle () {}
triangle(const point&p0,const point&p1,const point&p2){
   p[0] = p0, p[1] = p1, p[2] = p2;
   child[0] = child[1] = child[2] = 0;
}
bool inside(const point &pp) const {
   point a = p[0]-pp, b = p[1]-pp, c = p[2]-pp;
   return cross(a, b) >= 0 &&
    cross(b, c) >= 0 &&
    cross(c, a) >= 0;
}
};
triangle T[maxn*3]; int ct;
bool is_leaf ( int t ) {
return !T[t].child[0]&&!T[t].child[1]&&!T[t].child[2];
}
void add_edge ( edge a, edge b ) {
if ( a.t != -1 ) T[a.t].e[a.side] = b;
if ( b.t != -1 ) T[b.t].e[b.side] = a;
}
struct Triangulation {
Triangulation ( ) {
   int M = 1e5 * 3;//multiplicar el maximo valor por 3
   T[0]=triangle(point(-M,-M),point(M,-M),point(0,M));
   ct = 1;
}
int find ( int t, const point &p ) {
   while ( !is_leaf(t) ) {
     for ( int i = 0; i < 3; i ++ )
           if (T[t].child[i]&&T[T[t].child[i]].inside(p)){
             t = T[t].child[i]; break;
           }
   } return t;
}
void add_point ( const point &p ) {
   int t = find ( 0, p ), tab, tbc, tca;
   tab = ct;
   T[ct++] = triangle ( T[t].p[0], T[t].p[1], p );
   tbc = ct;
   T[ct++] = triangle ( T[t].p[1], T[t].p[2], p );
   tca = ct;
   T[ct++] = triangle ( T[t].p[2], T[t].p[0], p );
   add_edge ( {tab,0}, {tbc,1} );
   add_edge ( {tbc,0}, {tca,1} );
   add_edge ( {tca,0}, {tab,1} );
   add_edge ( {tab,2}, T[t].e[2] );
   add_edge ( {tbc,2}, T[t].e[0] );
   add_edge ( {tca,2}, T[t].e[1] );
   T[t].child[0] = tab; T[t].child[1] = tbc;
   T[t].child[2] = tca;
   flip ( tab, 2 ); flip ( tbc, 2 ); flip ( tca, 2 );
}
void flip ( int ti, int pi ) {
   int tj = T[ti].e[pi].t;
   int pj = T[ti].e[pi].side;
   if ( tj == -1 ) return;
   if (!incircle(T[ti].p[0],T[ti].p[1],
                              T[ti].p[2],T[tj].p[pj])) return;
   int tk = ct;
   T[ct++]=triangle(T[ti].p[(pi+1)%3],
                                    T[tj].p[pj],T[ti].p[pi]);
   int tl = ct;
   T[ct++] = triangle ( T[tj].p[(pj+1)%3],
                                                T[ti].p[pi], T[tj].p[pj] );
   add_edge ( {tk,0}, {tl,0} );
   add_edge ( {tk,1}, T[ti].e[(pi+2)%3] );
   add_edge ( {tk,2}, T[tj].e[(pj+1)%3] );
   add_edge ( {tl,1}, T[tj].e[(pj+2)%3] );
   add_edge ( {tl,2}, T[ti].e[(pi+1)%3] );
   T[ti].child[0] = tk, T[ti].child[1] = tl,
   T[ti].child[2] = 0;
   T[tj].child[0] = tk, T[tj].child[1] = tl,
   T[tj].child[2] = 0;
   flip ( tk, 1 ); flip ( tk, 2 );
   flip ( tl, 1 ); flip ( tl, 2 );
}
} delaunay;
void triangulate ( ) {
delaunay = Triangulation();
random_shuffle ( P+1, P+1+n );
for ( int i = 1; i <= n; i ++ )
   delaunay.add_point ( P[i] );
}
```

1.4.2. Pick Theorem O(n)

```
/*A = I + B/2 - 1:
  A = Area of the polygon
  I = Number of integer coordinates points inside
  B = Number of integer coordinates points on the boundary
  Polygon's vertex must have integer coordinates */
ll points_on_segment(const line &s){
   point p = s[0] - s[1];
   return __gcd(abs(p.x), abs(p.y));
}
pair<ll, ll> pick_theorem(polygon &P){
   ll A = area2(P), B = 0, I = 0;
   for (int i = 0, n = P.size(); i < n; ++i)
           B += points_on_segment({P[i], P[NEXT(i)]});
   A = abs(A);
   I = (A - B) / 2 + 1;
   return {I, B};// < points inside, points in boundary>
}
```

1.4.3. Primitives

```
/**1- Base element
   2- The traveling direction of the point (ccw)
   3- Intersection
   4- Distance.
   5- Polygon inclusion decision point
   6- Area of a polygon
   7- Scale a polygon
   8- triangulation possible non convex poly O(n^2)
   9-Convex hull (Andrew's Monotone Chain)
   10-Cutting of a convex polygon
   11-Convex polygon inclusion decision point
   12-Incircle
   13-Closest Pair Point
   14-Three Point Circle
   15-Circle_circle_intersect
   16-Tangents Point Circle
   17-Circle-Line-Intersection
   18-Centroid of a (possibly nonconvex) Polygon
   19-Point rotate **/
///----1-Base element----
struct point {
   db x, y;
   point ( db xx = 0, db yy = 0 ): x(xx), y(yy) { }
   point operator + ( const point &a ) const {
           return { x+a.x, y+a.y };
   }
   point operator - ( const point &a ) const {
           return { x-a.x, y-a.y };
   }
   point operator * ( const db &c ) const {
           return { x*c, y*c };
   }
   point operator * ( const point &p ) const {
           return { x*p.x - y*p.y, x*p.y + y*p.x };
   }
   point operator / ( const db &c ) const {
           return { x/c, y/c };
   }
   point operator / ( const point &a ) const {
        return point { x*a.x + y*a.y, y*a.x - x*a.y } /
           /*divide 2 complejos*/( a.x*a.x + a.y*a.y );
    }
   bool operator < ( const point &a ) const {
           if ( abs( x-a.x ) > eps )
                   return x+eps < a.x;
           return y+eps < a.y;
   }
};
typedef vector<point> polygon;
struct line : public vector<point> {
  line(const point &a, const point &b) {
    push_back(a); push_back(b);
  }
};
struct circle { point p; db r; };
db cross ( const point &a, const point &b ) {
   return a.x*b.y - a.y*b.x;
}
db dot ( const point &a, const point &b ) {
   return a.x*b.x + a.y*b.y;
}
db norm ( const point &p ) {
   return dot ( p, p );
}
db abs ( const point &p ) {
   return sqrt ( norm(p) );
}
db arg ( const point &p ) {
   return atan2 ( p.y, p.x );
}
point conj ( const point &p ) {
    return point { p.x, -p.y };
}
point crosspoint(const line &l, const line &m) {
  db A = cross(l[1] - l[0], m[1] - m[0]);
  db B = cross(l[1] - l[0], l[1] - m[0]);
  if (abs(A)<eps&&abs(B)<eps) return m[0];//same line
  if (abs(A)<eps)assert(false);//PRECONDITION NOT SATISFIED
  return m[0] + (m[1] - m[0])* B / A;
}
///---2-The traveling direction of the point------
int ccw(point a, point b, point c) {
  b = b-a; c = c-a;
  if (cross(b, c) > 0)return +1; // counter clockwise
  if (cross(b, c) < 0)return -1; // clockwise
  if (dot(b, c) < 0)  return +2; // c--a--b on line
  if (norm(b) < norm(c))return -2;// a--b--c on line
  return 0;
}
///----3-Intersection------
bool intersectLL(const line &l, const line &m) {
  return abs(cross(l[1]-l[0],m[1]-m[0]))>eps//non-parallel
         ||abs(cross(l[1]-l[0],m[0]-l[0]))<eps;//same line
}
bool intersectLS(const line &l, const line &s) {
  return cross(l[1]-l[0], s[0]-l[0])* // s[0] is left of l
        cross(l[1]-l[0],s[1]-l[0])<eps;//s[1] is right of l
}
bool intersectLP(const line &l, const point &p) {
  return abs(cross(l[1]-p, l[0]-p)) < eps;
}
bool intersectSS(const line &s, const line &t) {
  return ccw(s[0],s[1],t[0])*ccw(s[0],s[1],t[1]) <= 0 &&
         ccw(t[0],t[1],s[0])*ccw(t[0],t[1],s[1]) <= 0;
}
bool intersectSP(const line &s, const point &p) {
  return abs(s[0]-p)+abs(s[1]-p)-abs(s[1]-s[0])<eps;
}
```

```
109. ///---4-Distance----------------
110. point projection(const line &l, const point &p) {
111.   db t = dot(p-l[0], l[0]-l[1]) / norm(l[0]-l[1]);
112.   return l[0] + (l[0]-l[1])*t;
113. }
114. point reflection(const line &l, const point &p) {
115.   return p + point(2,0)*(projection(l, p) - p);
116. }
117. double distanceLP(const line &l, const point &p) {
118.   return abs(p - projection(l, p));
119. }
120. double distanceLL(const line &l, const line &m) {
121.   return intersectLL(l, m) ? 0 : distanceLP(l, m[0]);
122. }
123. double distanceLS(const line &l, const line &s) {
124.   if (intersectLS(l, s)) return 0;
125.   return min(distanceLP(l, s[0]), distanceLP(l, s[1]));
126. }
127. double distanceSP(const line &s, const point &p) {
128.   const point r = projection(s, p);
129.   if (intersectSP(s, r)) return abs(r - p);
130.   return min(abs(s[0] - p), abs(s[1] - p));
131. }
132. double distanceSS(const line &s, const line &t) {
133.   if (intersectSS(s, t)) return 0;
134.   return min(min(distanceSP(s, t[0]), distanceSP(s, t[1])),
135.              min(distanceSP(t, s[0]), distanceSP(t, s[1])));
136. }
137. ///----5-Polygon inclusion decision point----
138. #define curr(G, i) G[i]
139. #define next(G, i) G[(i+1)%G.size()]
140. enum { OUT, ON, IN };
141. int contains(const polygon &G, const point& p) {
142.   bool in = false;
143.   for (int i = 0; i < (int)G.size(); ++i) {
144.     point a = curr(G,i) - p, b = next(G,i) - p;
145.     if (a.y > b.y) swap(a, b);
146.     if (a.y <= 0 && 0 < b.y)
147.       if (cross(a, b) < 0) in = !in;
148.     if (cross(a, b) == 0 && dot(a, b) <= 0) return ON;
149.   }
150.   return in ? IN : OUT;
151. }
152. ///----6-Area of a polygon---------------------------
153. double area2(const polygon& G) {
154.   double A = 0;
155.   for (int i = 0; i < (int)G.size(); ++i)
156.     A += cross(curr(G, i), next(G, i));
157.   return A;
158. }
159. ///-----7-Scale a polygon---
160. #define prev(G,i) G[(i-1+G.size())%G.size()]
161. polygon shrink_polygon(const polygon &G, double len) {
162.   polygon res;
163.   for (int i = 0; i < (int)G.size(); ++i) {
164.     point a = prev(G,i), b = curr(G,i), c = next(G,i);
165.     point u = (b - a) / abs(b - a);
166.     double th = arg((c - b)/ u) * 0.5;
167.     res.push_back( b + u * point(-sin(th), cos(th))
168.                                 * len / cos(th) );
169.   }
170.   return res;
171. }
172. ///-----8-triangulation possibly non convex poly O(n^2)--
173. polygon make_triangle(const point&a,const point&b,
174.            const point&c){
175.   polygon ret(3);
176.   ret[0] = a; ret[1] = b; ret[2] = c;
177.   return ret;
178. }
179. bool triangle_contains(const polygon&tri,const point&p){
180.   return ccw(tri[0], tri[1], p) >= 0 &&
181.          ccw(tri[1], tri[2], p) >= 0 &&
182.          ccw(tri[2], tri[0], p) >= 0;
183. }
184. bool ear_Q(int i, int j, int k, const polygon& G) {
185.   polygon tri = make_triangle(G[i], G[j], G[k]);
186.   if (ccw(tri[0], tri[1], tri[2]) <= 0) return false;
187.   for (int m = 0; m < (int)G.size(); ++m)
188.     if (m != i && m != j && m != k)
189.       if (triangle_contains(tri, G[m]))
190.         return false;
191.   return true;
192. }
193. void triangulate(const polygon& G, vector<polygon>& t) {
194.   const int n = G.size();
195.   vector<int> l, r;
196.   for (int i = 0; i < n; ++i) {
197.     l.push_back( (i-1+n) % n );
198.     r.push_back( (i+1+n) % n );
199.   }
200.   int i = n-1;
201.   while ((int)t.size() < n-2) {
202.     i = r[i];
203.     if (ear_Q(l[i], i, r[i], G)) {
204.       t.push_back(make_triangle(G[l[i]], G[i], G[r[i]]));
205.       l[ r[i] ] = l[i];
206.       r[ l[i] ] = r[i];
207.     }
208.   }
209. }
210. ///---9-Convex_hull----------------------
211. vector<point> convex_hull(vector<point> ps) {
212.   int n = ps.size(), k = 0;
213.   sort(ps.begin(), ps.end());
214.   vector<point> ch(2*n);
215.   for (int i = 0; i < n; ch[k++] = ps[i++]) // lower-hull
216.     while (k >= 2 && ccw(ch[k-2], ch[k-1], ps[i]) <= 0)--k;
217.  for (int i = n-2,t = k+1;i>=0;ch[k++]=ps[i--])//upper-hull
218.     while (k >= t && ccw(ch[k-2], ch[k-1], ps[i]) <= 0)--k;
```

```
  ch.resize(k-1);
  return ch;
}
///---10-Cutting of a convex polygon-----------------
polygon convex_cut(const polygon& G, const line& l) {
  polygon Q;
  for (int i = 0; i < (int)G.size(); ++i) {
    point A = curr(G, i), B = next(G, i);
    if (ccw(l[0], l[1], A) != -1) Q.push_back(A);
    if (ccw(l[0], l[1], A)*ccw(l[0], l[1], B) < 0)
      Q.push_back(crosspoint(line(A, B), l));
  }
  return Q;
}
///---11-Convex polygon inclusion decision point-------
int convex_contains(const polygon &G, const point &p) {
   //G[0] must be the lowest right vertex
   int b = 1, e = G.size() - 1;
   while(b < e){
           int mid = (b + e) / 2;
           if(cross( G[0]-p, G[mid]-p) <= eps){
                   e = mid;
           }
           else b = mid + 1;
   }
   if(cross(G[b]-p,G[b-1]-p)<=eps&&
           cross(G[0]-p,G[b]-p)<=eps) if(b > 1
           or (G[0].y <= p.y + eps && p.y <= G[1].y + eps))
           return IN; // IN or ON
   return OUT;
}
///--------12-Incircle-------------------------------
bool incircle(point a, point b, point c, point p) {
  a = a-p; b = b-p; c = c-p;
  return norm(a) * cross(b, c)
       + norm(b) * cross(c, a)
       + norm(c) * cross(a, b) >= 0;
      // < : inside, = cocircular, > outside
}
///--13-closestPair-------------------------------
double closest_pair_points(vector<point> &P) {
   auto cmp = [](point a, point b) {
           return make_pair(a.y, a.x)
                           < make_pair(b.y, b.x);
   };
   int n = P.size();
   sort(P.begin(), P.end());
   set<point, decltype(cmp)> S(cmp);
   const double oo = 1e9; // adjust
   double ans = oo;
   for (int i = 0, ptr = 0; i < n; ++i) {
           while (ptr < i && abs(P[i].x - P[ptr].x) >= ans)
                   S.erase(P[ptr++]);
           auto lo = S.lower_bound(point(-oo,P[i].y-ans-eps));
           auto hi = S.upper_bound(point(-oo,P[i].y+ans+eps));
           for (decltype(lo) it = lo; it != hi; ++it)
                   ans = min(ans, abs(P[i] - *it));
           S.insert(P[i]);
   }
   return ans;
}
///----14-Three Point Circle-----------------------
point three_point_circle(const point&a,const point&b,
                                        const point&c){
  point x = (b - a)/norm(b-a), y = (c - a)/norm(c-a);
  return (y-x)/(conj(x)*y - x*conj(y)) + a;
}
///--15-Circle_circle_intersect--------------------
pair<point, point> circle_circle_intersect(const point&c1,
   const double& r1, const point& c2, const double& r2) {
  point A = conj(c2-c1);
  point B = ((c2-c1)*conj(c2-c1))*-1.0 + r2*r2-r1*r1 ;
  point C = (c2-c1)*r1*r1;
  point D = B*B-A*C*4.0;
  complex <db> q ( D.x, D.y );
  q = sqrt(q);
  D = { real(q), imag(q) };
  point z1 = (B*-1.0+D)/(A*2.0)+c1,
           z2 = (B*-1.0-D)/(A*2.0)+c1;
  return pair<point, point>(z1, z2);
}
///--16-Tangents Point Circle----------
vector<point> tangent(point p, circle c) {
    double D = abs(p - c.p);
    if (D + eps < c.r) return {};
    point t = c.p - p;
    double theta = asin( c.r / D );
    double d = cos(theta) * D;
    t = t / abs(t) * d;
    if ( abs(D - c.r) < eps ) return {p + t};
    point rot( cos(theta), sin(theta) );
    return {p + t * rot, p + t * conj(rot)};
}
///-17-Circle-Line-Intersection-----------------------
vector<point> intersectLC( line l, circle c ){
    point u = l[0] - l[1], v = l[0] - c.p;
    double a = dot(u,u), b = dot(u,v),
              cc = dot(v,v) - c.r * c.r;
    double det = b * b - a * cc;
    if ( det < eps ) return { };
    else return { l[0] + u * (-b + sqrt(det)) / a,
                  l[0] + u * (-b - sqrt(det)) / a };
}
///-18--Centroid of a (possibly nonconvex) Polygon
point centroid(const polygon &poly) {
   point c(0, 0);
   double scale = 3.0 * area2(poly);
   for (int i = 0, n = poly.size(); i < n; ++i) {
           int j = (i+1)%n;
           c=c+(poly[i]+poly[j])*(cross(poly[i],poly[j]));
```

```
    }
    return c / scale;
}
///-19-Point rotate--------------------------
inline point rotate(point A,double ang){//respect to origin
    double r =  sqrt(dot(A , A));
    double oang = atan2(A.y , A.x);
    return (point){cos(ang + oang),sin(ang + oang)} * r;
}
```

1.5. Graph

1.5.1. Dinic

```
const int MAXN = 2005;
const int INF = 1000000000;
struct edge {
    int a, b, cap, flow;
};
int n, s, t, d[MAXN], ptr[MAXN], q[MAXN];
vector<edge> e;
vector<int> g[MAXN];
void add_edge (int a, int b, int cap) {
    edge e1 = { a, b, cap, 0 };
    edge e2 = { b, a, 0, 0 };
    g[a].push_back ((int) e.size());
    e.push_back (e1);
    g[b].push_back ((int) e.size());
    e.push_back (e2);
}
bool bfs() {
    int qh=0, qt=0;
    q[qt++] = s;
    memset (d, -1, sizeof d);
    d[s] = 0;
    while (qh < qt && d[t] == -1) {
            int v = q[qh++];
            for (size_t i=0; i<g[v].size(); ++i) {
                    int id = g[v][i],
                            to = e[id].b;
                    if (d[to] == -1 && e[id].flow < e[id].cap) {
                            q[qt++] = to;
                            d[to] = d[v] + 1;
                    }
            }
    }
    return d[t] != -1;
}
int dfs (int v, int flow) {
    if (!flow)  return 0;
    if (v == t)  return flow;
    for (; ptr[v]<(int)g[v].size(); ++ptr[v]) {
            int id = g[v][ptr[v]],
                    to = e[id].b;
            if (d[to] != d[v] + 1)  continue;
            int pushed = dfs (to, min (flow, e[id].cap - e[id].flow));
            if (pushed) {
                    e[id].flow += pushed;
                    e[id^1].flow -= pushed;
                    return pushed;
            }
    }
    return 0;
}
int dinic( )
{
     int flow = 0;
    for (;;) {
            if (!bfs())  break;
            memset (ptr, 0, sizeof ptr);
            while (int pushed = dfs (s, INF))
                    flow += pushed;
    }
     return flow;
}
///para multicasos limpiar el vector g desde 1->cant nodos y el vector e
```

1.5.2. Dominator Tree O((N+M)logN)

```
struct graph{
    int n;
    vector<vector<int> > adj, radj, to;
    graph(int n) : n(n), adj(n), radj(n), to(n) {}
    void add_edge(int src, int dst){
            adj[src].push_back(dst);
            radj[dst].push_back(src);
    }
    vector<int> rank, semi, low, anc;
    int eval(int v){
            if (anc[v] < n && anc[anc[v]] < n){
                    int x = eval(anc[v]);
                    if (rank[semi[low[v]]] > rank[semi[x]])
                            low[v] = x;
                    anc[v] = anc[anc[v]];
            }
            return low[v];
    }
    vector<int> prev, ord;
    void dfs(int u){
            rank[u] = ord.size();
            ord.push_back(u);
            for (int i = 0; i < (int) adj[u].size(); ++i){
                    int v = adj[u][i];
                    if (rank[v] < n)
                            continue;
                    dfs(v);
                    prev[v] = u;
            }
    }
    vector<int> idom; // idom[u] is an immediate dominator of u
    void dominator_tree(int r){
            idom.assign(n, n);
```

```
            prev = rank = anc = idom;
            semi.resize(n);
            for (int i = 0; i < n; ++i)
                    semi[i] = i;
            low = semi;
            ord.clear();
            dfs(r);
            vector<vector<int> > dom(n);
            for (int x = (int) ord.size() - 1; x >= 1; --x){
                    int w = ord[x];
                    for (int j = 0; j < (int) radj[w].size(); ++j){
                            int v = radj[w][j];
                            int u = eval(v);
                            if (rank[semi[w]] > rank[semi[u]])
                                    semi[w] = semi[u];
                    }
                    dom[semi[w]].push_back(w);
                    anc[w] = prev[w];
                    for (int i=0;i<(int)dom[prev[w]].size();++i){
                            int v = dom[prev[w]][i];
                            int u = eval(v);
                            idom[v] = (rank[prev[w]] > rank[semi[u]]?
                                                            u : prev[w]);
                    }
                    dom[prev[w]].clear();
            }
            for (int i = 1; i < (int) ord.size(); ++i){
                    int w = ord[i];
                    if (idom[w] != semi[w])
                            idom[w] = idom[idom[w]];
            }
    }
    vector<int> dominators(int u){
            vector<int> S;
            for (; u < n; u = idom[u])
                    S.push_back(u);
            return S;
    }
    void tree(  ){
            for (int i = 0; i < n; ++i){
                    if (idom[i] < n)
                            to[ idom[i] ].push_back( i );
            }
    }
};
```

1.5.3. DSU on Tree

```
int cnt[maxn];
bool big[maxn];
void add(int v, int p, int x){
    cnt[ col[v] ] += x;
    for(auto u: g[v])
        if(u != p && !big[u])
            add(u, v, x)
}
void dfs(int v, int p, bool keep){
    int mx = -1, bigChild = -1;
    for(auto u : g[v])
       if(u != p && sz[u] > mx)
          mx = sz[u], bigChild = u;
    for(auto u : g[v])
        if(u != p && u != bigChild)
            dfs(u, v, 0);
    if(bigChild != -1)
        dfs(bigChild, v, 1), big[bigChild] = 1;
    add(v, p, 1);
    //now You can answer the queries easily.
    if(bigChild != -1)
        big[bigChild] = 0;
    if(keep == 0)//erase everything
        add(v, p, -1);
}
```

1.5.4. flow_with_lower_bound

```
Flow with lower bound
Complexity: O(n^2 m)
template<typename T>
struct dinic{
struct edge{
   int src, dst;
   T low, cap, flow;
   int rev;
};
int n;
vector<vector<edge>> adj;
dinic(int n) : n(n), adj(n + 2) {}
void add_edge(int src, int dst, T low, T cap)      {
   adj[src].push_back({ src, dst, low, cap, 0, (int) adj[dst].size() });
   if (src == dst)
           adj[src].back().rev++;
   adj[dst].push_back({ dst, src, 0, 0, 0, (int) adj[src].size() - 1 });
}
vector<int> level, iter;
T augment(int u, int t, T cur)     {
   if (u == t)
           return cur;
   for (int &i = iter[u]; i < (int) adj[u].size(); ++i){
           edge &e = adj[u][i];
           if (e.cap - e.flow > 0 && level[u] > level[e.dst]){
                   T f = augment(e.dst, t, min(cur, e.cap - e.flow));
                   if (f > 0){
                           e.flow += f;
                           adj[e.dst][e.rev].flow -= f;
                           return f;
                   }
           }
   }
   return 0;
}
int bfs(int s, int t){
```

```
    level.assign(n + 2, n + 2);
    level[t] = 0;
    queue<int> Q;
    for (Q.push(t); !Q.empty(); Q.pop()){
            int u = Q.front();
            if (u == s)
                    break;
            for (edge &e : adj[u]){
                    edge &erev = adj[e.dst][e.rev];
                    if (erev.cap - erev.flow > 0
                            && level[e.dst] > level[u] + 1){
                            Q.push(e.dst);
                            level[e.dst] = level[u] + 1;
                    }
            }
    }
    return level[s];
}
const T oo = numeric_limits<T>::max();
T max_flow(int source, int sink){
    vector<T> delta(n + 2);

    for (int u = 0; u < n; ++u) // initialize
            for (auto &e : adj[u]){
                    delta[e.src] -= e.low;
                    delta[e.dst] += e.low;
                    e.cap -= e.low;
                    e.flow = 0;
            }
    T sum = 0;
    int s = n, t = n + 1;
    for (int u = 0; u < n; ++u){
            if (delta[u] > 0){
                    add_edge(s, u, 0, delta[u]);
                    sum += delta[u];
            }
            else if (delta[u] < 0)
                    add_edge(u, t, 0, -delta[u]);
    }
    add_edge(sink, source, 0, oo);
    T flow = 0;
    while (bfs(s, t) < n + 2){
            iter.assign(n + 2, 0);
            for (T f; (f = augment(s, t, oo)) > 0;)
                    flow += f;
    }
    if (flow != sum)
            return -1; // no solution
    for (int u = 0; u < n; ++u)
            for (auto &e : adj[u]){
                    e.cap += e.low;
                    e.flow += e.low;
                    edge &erev = adj[e.dst][e.rev];
                    erev.cap -= e.low;
                    erev.flow -= e.low;
            }
    adj[sink].pop_back();
    adj[source].pop_back();
    while (bfs(source, sink) < n + 2){
            iter.assign(n + 2, 0);
            for (T f; (f = augment(source, sink, oo)) > 0;)
                    flow += f;
    } // level[u] == n + 2 ==> s-side

    return flow;
}
};
```

1.5.5. HopcroftâKarp Bipartite Matching O(Msqrt(N))

```
const int MAXV = 1001;
const int MAXV1 = 2*MAXV;
vector<int> ady[MAXV];
int D[MAXV1],Mx[MAXV], My[MAXV];
bool BFS(){
    int u, v, i, e;
    queue<int> cola;
    bool f = 0;
    for (i = 0; i < N+M; i++) D[i] = 0;
    for (i = 0; i < N; i++)
        if (Mx[i] == -1) cola.push(i);
    while (!cola.empty()){
        u = cola.front(); cola.pop();
        for (e = ady[u].size()-1; e >= 0; e--) {
            v = ady[u][e];
            if (D[v + N]) continue;
            D[v + N] = D[u] + 1;
            if (My[v] != -1){
                D[My[v]] = D[v + N] + 1;
                cola.push(My[v]);
            }else f = 1;
        }
    }
    return f;
}
int DFS(int u){
    for (int v, e = ady[u].size()-1; e >=0; e--){
        v = ady[u][e];
        if (D[v+N] != D[u]+1) continue;
        D[v+N] = 0;
        if (My[v] == -1 || DFS(My[v])){
            Mx[u] = v;  My[v] = u;  return 1;
        }
    }
    return 0;
}
int Hopcroft_Karp(){
    int i, flow = 0;
    for (i = max(N,M); i >=0; i--) Mx[i] = My[i] = -1;
    while (BFS())
        for (i = 0; i < N; i++)
```

```
            if (Mx[i] == -1 && DFS(i))
                ++flow;
     return flow;
}
```

1.5.6. Hungarian O(N^3)

```
#define MAXN 300
int N,A[MAXN+1][MAXN+1],p,q, oo = 1 <<30;
int fx[MAXN+1],fy[MAXN+1],x[MAXN+1],y[MAXN+1];
int hungarian(){
    memset(fx,0,sizeof(fx));
    memset(fy,0,sizeof(fy));
    memset(x,-1,sizeof(x));
    memset(y,-1,sizeof(y));
    for(int i = 0; i < N; ++i)
        for(int j = 0; j < N; ++j) fx[i] = max(fx[i],A[i][j]);
    for(int i = 0; i < N; ){
        vector<int> t(N,-1), s(N+1,i);
        for(p = q = 0; p <= q && x[i]<0; ++p)
            for(int k = s[p], j = 0; j < N && x[i]<0; ++j)
                if (fx[k]+fy[j]==A[k][j] && t[j]<0)
                {
                    s[++q]=y[j];
                    t[j]=k;
                    if(s[q]<0)
                        for(p=j; p>=0; j=p)
                            y[j]=k=t[j], p=x[k], x[k]=j;
                }
        if (x[i]<0){
            int d = oo;
            for(int k = 0; k < q+1; ++k)
                for(int j = 0; j < N; ++j)
                    if(t[j]<0) d=min(d,fx[s[k]]+fy[j]-A[s[k]][j]);
            for(int j = 0; j < N; ++j) fy[j]+=(t[j]<0?0:d);
            for(int k = 0; k < q+1; ++k) fx[s[k]]-=d;
        }
        else ++i;
    }
    int ret = 0;
    for(int i = 0; i < N; ++i) ret += A[i][x[i]];
    return ret;
}
```

1.5.7. MaxFlowMinCost

```
const int INF = 1e9;
typedef pair <int, int> par;
namespace MCF {
struct edge {
    int dest, cap, flow, cost, next;

    edge() {}

    edge(int dest, int cap, int cost, int next) :
            dest(dest), cap(cap), cost(cost), next(next) {
            flow = 0;
    }
};
const int
    MAXV = 10e3 + 5,
    MAXE = 10e4 + 5;
int nodes, source, sink, size;
edge g[MAXE];
int first[MAXV], nLast[MAXV], eLast[MAXV], cst[MAXV];

void init(int _nodes, int _source, int _sink) {
    size = 0, nodes = _nodes, source = _source;
    sink = _sink;

    for(int i = 0; i < nodes; i++)
            first[i] = -1;
}
void addEdge(int u, int v, int c, int cost) {
    g[size] = edge(v, c, +cost, first[u]);
    first[u] = size++;
    g[size] = edge(u, 0, -cost, first[v]);
    //bidirectional, put zero if only directed edges!
    first[v] = size++;
}
par maxFlowMinCost() {
    int flow = 0, flowCost = 0;
    while(true) {
            priority_queue <par> q;

            for(int i = 0; i < nodes; i++)
                    cst[i] = INF;

            cst[source] = 0;
            q.push(make_pair(0, source));

            while(!q.empty()) {
                    int u = q.top().second;
                    int c = -q.top().first;
                    q.pop();

                    if(u == sink)
                            break;

                    for(int k = first[u]; k != -1; k = g[k].next) {
                            int v = g[k].dest;
                            int w = g[k].cost + c;

                            if(g[k].cap > g[k].flow && w < cst[v]) {
                                    cst[v] = w;
                                    nLast[v] = u;
                                    eLast[v] = k;
                                    q.push(make_pair(-w, v));
                            }
                    }
            }

```

```
67.             if(cst[sink] == INF)
68.                     break;
69.
70.             int push = INF;
71.             for(int u = sink; u != source; u = nLast[u])
72.                     push = min(push, g[eLast[u]].cap - g[eLast[u]].flow);
73.
74.             flow += push;
75.             flowCost += cst[sink] * push;
76.
77.             for(int u = sink; u != source; u = nLast[u]) {
78.                     g[eLast[u]].flow += push;
79.                     g[eLast[u] ^ 1].flow -= push;
80.             }
81.     }
82.
83.     return make_pair(flow, flowCost);
84. }
85. }
86.
```

1.5.8. Minimum Arborescences O(MlogN)

```
1. template<typename T>
2. struct minimum_aborescense{
3.     struct edge{
4.             int src, dst;
5.             T weight;
6.     };
7.     vector<edge> edges;
8.     void add_edge(int u, int v, T w){
9.             edges.push_back({ u, v, w });
10.    }
11.    T solve(int r){
12.            int n = 0;
13.            for (auto e : edges)
14.                    n = max(n, max(e.src, e.dst) + 1);
15.            int N = n;
16.            if( N == 0 ) return 0;
17.            for (T res = 0;;){
18.             vector<edge> in(N, { -1, -1, numeric_limits<T>::max() });
19.                    vector<int> C(N, -1);
20.                    for (auto e : edges)
21.                            if (in[e.dst].weight > e.weight)
22.                                    in[e.dst] = e;
23.                    in[r] = {r, r, 0};
24.                    for (int u = 0; u < N; ++u){
25.                            if (in[u].src < 0)
26.                                    return numeric_limits<T>::max();
27.                            res += in[u].weight;
28.                    }
29.                    vector<int> mark(N, -1);
30.                    int index = 0;
31.                    for (int i = 0; i < N; ++i)     {
32.                            if (mark[i] != -1)      continue;
33.                            int u = i;
34.                            while (mark[u] == -1){
35.                                    mark[u] = i;
36.                                    u = in[u].src;
37.                            }
38.                            if (mark[u] != i || u == r)
39.                                    continue;
40.                            for(int v=in[u].src;u!=v;v=in[v].src)
41.                                    C[v] = index;
42.                            C[u] = index++;
43.                    }
44.                    if (index == 0) return res;
45.                    for (int i = 0; i < N; ++i)
46.                            if (C[i] == -1) C[i] = index++;
47.                    vector<edge> next;
48.                    for (auto &e : edges)
49.                if(C[e.src]!=C[e.dst]&&C[e.dst]!=C[r])
50.                                    next.push_back({C[e.src], C[e.dst],
51.                                            e.weight-in[e.dst].weight});
52.                    edges.swap(next);
53.                    N = index;
54.                    r = C[r];
55.            }
56.    }
57. };
```

1.5.9. MO's Algorithm on Tree

```
1. const int maxn=40011,maxq=100011;
2. int BLOCK;
3. struct node{
4.     int l, r, i, sp, ans;
5. }Q[maxq];
6. int lvl[maxn], p[maxn][16], st[maxn], en[maxn], id[2*maxn], occ[maxn];
7. int ans, w[maxn];
8. map<int, int> HASH;
9. int cnt[maxn];
10. vector<int> g[maxn];
11. int ti;
12. bool f(node a, node b){
13.     if (a.l/BLOCK != b.l/BLOCK)
14.             return a.l < b.l;
15.     return a.r < b.r;
16. }
17. bool gg(node a, node b){
18.     return a.i < b.i;
19. }
20. void dfs(int u, int p){
21.     st[u] = ++ti;
22.     id[ti] = u;
23.     int v;
24.     for(auto v: g[u]){
25.             if (v == p)
26.                     continue;
27.             lvl[v] = lvl[u]+1;
28.             ::p[v][0] = u;
29.             dfs(v, u);
```

```
    }
    en[u] = ++ti;
    id[ti] = u;
}
void add(int node){
    occ[node]++;
    cnt[w[node]]++;
    if (occ[node] == 2){
            cnt[w[node]] -= 2;
            if (cnt[w[node]] == 0)
                    ans--;
    }
    else if (cnt[w[node]] == 1) ans++;
}
void del(int node){
    int wt = w[node];
    occ[node]--;

    if (occ[node] == 1){
            cnt[wt]++;
            if (cnt[wt] == 1)
                    ans++;
            return;
    }
    cnt[wt]--;
    if (cnt[wt] == 0) ans--;
}
    int n, m, i, j, u, v;
    ans = ti = 0;
    cin>>n>>m;
    BLOCK = sqrt(n);
    int no = 0;
    HASH.clear();
    //compress coordinates
    for( int i = 1; i <= n; i++ ){
            cin>>w[i];
            if (HASH.find(w[i]) == HASH.end())
                    HASH[w[i]] = ++no;
            w[i] = HASH[w[i]];

    }
    //read de graph
    for( int i = 1; i < n; i++ ){
            cin>>u>>v;
            g[u].push_back(v);
            g[v].push_back(u);
    }

    lvl[1] = 0;
    memset(cnt, 0, sizeof(cnt));
    memset(occ, 0, sizeof(occ));
    memset(p, -1, sizeof(p));
    //compute the necesary values
    dfs(1, 0);
    //compute lca
    for(i=1; i<16; i++)
    for(j=1; j<=n; j++)
            if( p[j][i-1] != -1)
                    p[j][i] = p[p[j][i-1]][i-1];

     //read th queries
    for( int i = 1; i <= m; i++){
            Q[i].i = i;
            Q[i].sp = -1;
            cin>>u>>v;
            if (lvl[u] < lvl[v])
                    swap(u, v);
            int w = lca(u, v);
            //case 1
            if (w == v){
                    Q[i].l = st[v];
                    Q[i].r = st[u]+1;
            }//case 2
            else{
                    if (st[v] > en[u]){
                            Q[i].l = en[u];
                            Q[i].r = st[v]+1;
                    }
                    else{
                            Q[i].l = en[v];
                            Q[i].r = st[u]+1;
                    }
                    // Special case: We have to consider 'w' separately.
                    Q[i].sp = w;
            }
    }
    sort(Q, Q+m, f);
    int currL = 0, currR = 0, L, R;
    for( int i = 1; i <= m; i++ ){
            L = Q[i].l, R = Q[i].r;
            while (currL < L){
                    del(id[currL]);
                    currL++;
            }
            while (currL > L){
                    add(id[currL-1]);
                    currL--;
            }
            while (currR < R){
                    add(id[currR]);
                    currR++;
            }
            while (currR > R){
                    del(id[currR-1]);
                    currR--;
            }
            Q[i].ans = ans;
            if (Q[i].sp != -1){
                    if (cnt[w[Q[i].sp]] == 0)
                            Q[i].ans = ans+1;
```

```
            }
    }
```

1.5.10. Punto de Art. y Bridges O(N)

```
void bridges_PtoArt ( int nod ){
    Td[nod] = low[nod] = ++ k;
    for( auto num : V[nod] ){
        int newn = G[num].nextn( nod );
        if( G[num].band ) continue;
        G[num].band = true;
        if( Td[newn] ){
            low[nod] = min( low[nod], Td[newn] );
            continue;
        }
        bridges_PtoArt( newn );
        low[nod] = min( low[nod], low[newn] );
        if(Td[nod] < low[newn])
            puente.push(par( nod, newn ));
        if( (Td[nod] == 1 && Td[newn] > 2 ) ||
            ( Td[nod] != 1 && Td[nod] <= low[newn] ) )
            Punto_art[nod] = true;
    }
}
```

1.5.11. Stable Marriage

```
typedef vector<int> vi; typedef vector<vi> vvi;
#define rep(i,a,b) for ( __typeof(a) i=(a); i<(b); ++i)
vi stable_marriage(int n, int **m, int **w){
    queue<int> q;
    vi at(n, 0), eng(n, -1), res(n, -1); vvi inv(n, vi(n));
    rep(i,0,n) rep(j,0,n) inv[i][w[i][j]] = j;
    rep(i,0,n) q.push(i);
    while (!q.empty()) {
        int curm = q.front(); q.pop();
        for (int &i = at[curm]; i < n; i++) {
            int curw = m[curm][i];
            if (eng[curw] == -1) { }
            else if (inv[curw][curm] < inv[curw][eng[curw]])
                q.push(eng[curw]);
            else continue;
            res[eng[curw] = curm] = curw, ++i; break;
        }
    }
    return res;
}
```

1.5.12. steiner_tree

```
//el steiner tree es un caso especiel de mst donde en este no se abarca
//necesariamente todo el conjunto de nodos sino un subconjunto estpecifico
//de estos permitiendo el uso de algunos nodos intermedios
//Complejidad (2^cTerminales)*N^2 cTerminales->numero de nodos terminales
//N->numero de nodos totales incluidos los terminales.
for( int i = 1; i < (1 << CTerminal); i++ )
   for( int j = 0; j < N; j++ )
           dp[i][j] = oo;
//importante caso base cuando el nodo terminal es la raiz de su arbol
//mask = 1 << i i->id del nodo con respecto a los terminales,
//id_terminal = id del nodo con respecto a todos los nodos
for( int i = 0; i < cTerminal; i++ )
   dp[1 << i][id_terminal] = 0;
//dp para computar el minimo steiner tree
//los estados son teniendo a root como raiz del arbol y a mask como los
//nodos terminales que
//estan en el arbol
for( int mask = 1; mask < (1 << cTerminal); mask++ ){
for( int root = 0; root < N; root ++ ){
for( int smask = mask; smask; smask = (smask-1)&mask ){
dp[mask][root] = min(dp[mask][root], dp[smask][root]+dp[mask^smask][root]);
}
//ciclo para mover la raiz del arbol
if( dp[mask][root] < oo ){
   for( int j = 0; j < N; j++ )
           dp[mask][j] = min(dp[mask][j],dp[mask][root] + C[root][j]);
  }
 }
}
//sol = min(dp[1 << Cterminal][i])
```

1.5.13. StoerWagner O(N^3)

```
//maximo flujo seleccionando la mejor fuente y mejor sumidero
int G[MAXN][MAXN], w[MAXN], N;
bool A[MAXN], merged[MAXN];
int StoerWagner(int n){
    int best = 1e8;
    for(int i=1;i<n;++i) merged[i] = 0;
    merged[0] = 1;
    for(int phase=1;phase<n;++phase){
        A[0] = 1;
        for(int i=1;i<n;++i){
            if(merged[i]) continue;
            A[i] = 0;
            w[i] = G[0][i];
        }
        int prev = 0,next;
        for(int i=n-1-phase;i>=0;--i){
            // hallar siguiente vertice que no esta en A
            next = -1;
            for(int j=1;j<n;++j)
                if(!A[j] && (next==-1 || w[j]>w[next]))
                    next = j;
            A[next] = true;
            if(i>0){
                prev = next;
                // actualiza los pesos
                for(int j=1;j<n;++j) if(!A[j])
                   w[j] += G[next][j];
            }
        }
        if(best>w[next]) best = w[next];
        for(int i=0;i<n;++i){// mezcla s y t
```

```
            G[i][prev] += G[next][i];
            G[prev][i] += G[next][i];
        }
        merged[next] = true;
    }
    return best;
}
```

1.6. Number Theory

1.6.1. Algoritmo Shanka-Tonelli (x^2 = a(mod p))

```
//devuelve x (mod p) tal que x^2 = a (mod p)
long long solve_quadratic( long long a, int p ){
    if( a == 0 ) return 0;
    if( p == 2 ) return a;
    if( powMod(a,(p-1)/2, p) != 1 ) return -1;
    int phi = p-1, n = 0, k = 0, q = 0;
    while( phi%2 == 0 ) phi/=2, n ++;
    k = phi;
    for( int j = 2; j < p; j ++ )
        if( powMod( j, (p-1)/2, p ) == p-1 ){
          q = j; break;
        }
    long long t = powMod( a, (k+1)/2, p );
    long long r = powMod( a, k, p );
    while( r != 1 ){
        int i = 0, v = 1;
        while( powMod( r, v, p ) != 1 ) v *= 2, i ++;
        long long e = powMod( 2, n-i-1, p );
        long long u = powMod( q, k*e, p );
        t = (t*u)%p;
        r = (r*u*u)%p;
    }
    return t;
}
```

1.6.2. Extended GCD (ax+by = gcd(a,b))

```
//devuelve x,y tal que ax+by = gcd(a,b)
int64 extended_euclid( int64 a, int64 b, int64& x, int64& y ) {
  int64 g = a;
  x = 1, y = 0;
  if ( b != 0 ) {
    g = extended_euclid( b, a % b, y, x );
    y -= ( a / b ) * x;
  }
  return g;
}
```

1.6.3. Fast Modulo Transform O(NlogN)

```
const int mod = 167772161;
// so the algorithm works until n = 2 ^17 = 131072
const int G = 3; // primitive root
//const int MOD = 1073872897 = 2  ^ 30 + 2 ^ 17 + 1, g = 7
// another good choice is MOD = 167772161 = 2^27+2^25+1, g = 3
// a bigger choice would be MOD = 3221225473 = 2^31+2^30+1, g = 5
// but it requires unsigned long long for multiplications
// n must be a power of two
// sign = 1
// sign = -1
// fast modulo transform
//   (1) n = 2^k < 2^23
//   (2) only predetermined mod can be used
//   (3) Inverso Modular */
void fmt(vector<ll> &x, int sign = +1){
   int n = x.size();
   for (int i = 0, j = 1; j < n - 1; ++j){
           for (int k = n >> 1; k > (i ^= k); k >>= 1);
           if (j < i) swap(x[i], x[j]);
   }
   ll h = pow(G, (mod - 1) / n, mod);
   if (sign < 0) h = inv(h, mod);
   for (int m = 1; m < n; m *= 2){
           ll w = 1, wk = pow(h, n / (2 * m), mod);
           for (int i = 0; i < m; ++i){
                   for (int j = i; j < n; j += 2 * m){
                           ll u = x[j], d = ( x[j + m] * w ) % mod;
                           x[j] = (u + d)%mod;
                           x[j + m] = (u - d + mod)%mod;
                   }
                   w = w * wk % mod;
           }
   }
   if (sign < 0){
           ll n_inv = inv(n, mod);
           for (auto &a : x) a = (a * n_inv) % mod;
   }
}
```

1.6.4. fft

```
const int MN = 1e5+2;

typedef long long ll;
typedef complex<double> base;
double PI = acos(-1);

void fft(vector<base> &a, bool invert){
    int n = (int) a.size();
    for (int i=1, j=0; i < n; i++){
        ll bit = n >> 1;
        for(; j >= bit; bit >>= 1)  j -= bit;
        j += bit;
        if(i < j)  swap(a[i], a[j]);
    }
    for(int len = 2; len <= n; len <<= 1){
        double ang = 2 * PI / len * (invert ? -1 : 1);
        base wlen (cos(ang), sin(ang));
        for(int i = 0; i < n; i += len){
            base w(1);
            for(int j = 0; j < len/2; ++j){
                base u = a[i+j], v = a[i + j + len/2] * w;
                a[i + j] = u + v;
```

```
                a[i + j + len/2] = u - v;
                w *= wlen;
            }
        }
    }
    if(invert)
        for(int i = 0; i < n; ++i) a[i] /= n;
}

void multiply(const vector<ll> &a, const vector<ll> &b, vector<ll> &res){
    vector<base> fa(a.begin(), a.end()), fb(b.begin(), b.end());
    size_t n = 1;
    while(n < max(a.size(), b.size())) n <<= 1;
    n <<= 1;
    fa.resize(n), fb.resize(n);

    fft(fa, false), fft(fb, false);
    for(size_t i = 0; i < n; ++i) fa[i] *= fb[i];

    fft(fa, true);

    res.resize(n);
    for(size_t i = 0; i < n; ++i) res[i] = ll(fa[i].real() + 0.5);
}
```

1.6.5. Find a primitive root of a prime number

```
// Assuming the Riemnan Hypothesis it runs in O(log^6(p)*sqrt(p))
int generator (int p){
    vector<int> fact;
    int phi = p-1, n = phi;
    for (int i=2; i*i<=n; ++i)
        if (n % i == 0){
            fact.push_back (i);
            while (n % i == 0)
                n /= i;
        }
    if (n > 1) fact.push_back (n);
    for (int res=2; res<=p; ++res){
        bool ok = true;
        for (size_t i=0; i<fact.size() && ok; ++i)
            ok &= powmod (res, phi / fact[i], p) != 1;
        if (ok) return res;
    }
    return -1;
}
```

1.6.6. Floyds Cycle-Finding algorithm

```
par find_cycle() {
    int t = f(x0), h = f(t), mu = 0, lam = 1;
    while (t != h) t = f(t), h = f(f(h));
    h = x0;
    while (t != h) t = f(t), h = f(h), mu++;
    h = f(t);
    while (t != h) h = f(h), lam++;
    return par(mu, lam);
}
```

1.6.7. Gauss

```
//   (1) solving systems of linear equations (AX=B)
//   (2) inverting matrices (AX=I)
//   (3) computing determinants of square matrices
// Running time: O(n^3)
// INPUT:    a[][] = an nxn matrix   b[][] = an nxm matrix
// OUTPUT:   X = an nxm matrix (stored in b[][])
//                            A^{-1} = an nxn matrix (stored in a[][])
//                            returns determinant of a[][]
1539171873
const double EPS = 1e-10;
typedef vector<int> VI;
typedef double T;
typedef vector<T> VT;
typedef vector<VT> VVT;
T GaussJordan(vector<vector<T> > &a, vector<vector<T> > &b) {
const int n = a.size();
const int m = b[0].size();
vector<int> irow(n), icol(n), ipiv(n);
T det = 1;
for (int i = 0; i < n; i++) {
int pj = -1, pk = -1;
for (int j = 0; j < n; j++)
   if (!ipiv[j])
           for (int k = 0; k < n; k++)
                   if (!ipiv[k])
                           if (pj == -1 || fabs(a[j][k]) > fabs(a[pj][pk]))
{
                                   pj = j;
                                   pk = k;
                           }
                           debug(a);
   if (fabs(a[pj][pk]) < EPS) {
           cerr << "Matrix is singular." << endl;
           exit(0);
   }
   ipiv[pk]++;
   swap(a[pj], a[pk]);
   swap(b[pj], b[pk]);
   if (pj != pk)
           det *= -1;
   irow[i] = pj;
   icol[i] = pk;
   T c = 1.0 / a[pk][pk];
   det *= a[pk][pk];
   a[pk][pk] = 1.0;
   for (int p = 0; p < n; p++)
           a[pk][p] *= c;
   for (int p = 0; p < m; p++)
           b[pk][p] *= c;
   for (int p = 0; p < n; p++)
           if (p != pk) {
                   c = a[p][pk];
```

```
                    a[p][pk] = 0;
                    for (int q = 0; q < n; q++)
                            a[p][q] -= a[pk][q] * c;
                    for (int q = 0; q < m; q++)
                            b[p][q] -= b[pk][q] * c;
            }
}
for (int p = n - 1; p >= 0; p--)
   if (irow[p] != icol[p]) {
            for (int k = 0; k < n; k++)
                    swap(a[k][irow[p]], a[k][icol[p]]);
}
return det;
}
```

1.6.8. Inverso de Polinomio

```
Inverso de Polinomio
 pol inverse(pol &a){
 assert(a[ 0] != 0);
 pol r(1, modexp(a[0], mod - 2));
 while (r.size() < a.size()){
 int csize = 2 * (int)r.size();
 r.resize(csize);
 pol tr = r;
 tr.resize( 2 * csize);
 pol tf( 2 * csize);
 for (int i = 0; i < csize; ++i) tf[i] = a[i];
 fft(tr, +1), fft(tf, +1);
 for (int i = 0; i < 2 * csize; ++i)
 tr[i] = ((1LL*tr[i]*tr[i])%mod*tf[i])%mod;
 fft(tr, -1);
 for (int i = 0; i < csize; ++i){
 add(r[i], r[i]);
 sub(r[i], tr[i]);
 }
 }
 return r;
 }
 pol sqrt(pol &a){
 // rt^2 = a[0] (mod)
 // this only works if a[0] = 1
 int rt = 1;
 pol r( 1, rt);
 while (r.size() < a.size()){
 int csize = 2 * (int)r.size();
 r.resize(csize);
 pol tf(csize);
 for (int i = 0; i < csize; ++i) tf[i] = a[i];
 pol ir = inverse(r);
 multiply(tf, ir);
 for (int i = 0; i < csize; ++i){
 add(r[i], tf[i]);
 r[i] = ( 1LL*r[i]*i2)% mod;
 }
```

```
 }
 return r;
 }
```

1.6.9. Inverso Modular

```
//mcd(n,m)==1
ll inv(ll b, ll M){
   ll u = 1, x = 0, s = b, t = M;
   while( s ){
           ll q = t / s;
           swap(x -= u * q, u);
           swap(t -= s * q, s);
   }
   return (x %= M) >= 0 ? x : x + M;
}
```

1.6.10. Josephus

```
// n-cantidad de personas, m es la longitud del salto.
// comienza en la k-esima persona.
ll josephus(ll n, ll m, ll k) {
 ll x = -1;
 for (ll i = n - k + 1; i <= n; ++i) x = (x + m) % i;
 return x;
}
ll josephus_inv(ll n, ll m, ll x){
 for (ll i = n;; i--){
 if (x == i) return n - i;
 x = (x - m % i + i) % i;
 }
 return -1;
}
```

1.6.11. Linear Recurrence Solver O(N^2logK)

```
/* x[i+n] = a[0] x[i] + a[1] x[i+1] + ... + a[n-1] x[i+n-1]
   with initial solution x[0], x[1], ..., x[n-1]
   Complexity: O(n^2 log k) time, O(n log k) space */
ll linear_recurrence(vector<ll> a, vector<ll> x, ll k){
   int n = a.size();
   vector<ll> t(2 * n + 1);
   function<vector<ll>(ll)> rec = [&](ll k){
           vector<ll> c(n);
           if (k < n) c[k] = 1;
           else{
                   vector<ll> b = rec(k / 2);
                   fill(t.begin(), t.end(), 0);
                   for (int i = 0; i < n; ++i)
                           for (int j = 0; j < n; ++j){
                                   t[i+j+(k&1)] += (b[i]*b[j])%mod;
                                   t[i+j+(k&1)] %= mod;
                           }
                   for (int i = 2*n-1; i >= n; --i)
                           for (int j = 0; j < n; ++j){
                                   t[i-n+j] += (a[j]*t[i])%mod;
                                   t[i-n+j] %= mod;
```

```
                        }
                for (int i = 0; i < n; ++i)
                        c[i] = t[i];
        }
        return c;
};
vector<ll> c = rec(k);
ll ans = 0;
for (int i = 0; i < x.size(); ++i){
        ans += (c[i] * x[i])%mod;
        ans %= mod;
}
return ans;
}
```

1.6.12. linear_congruences

```
/*
   Solve x=ai(mod mi), for any i and j, (mi,mj)|ai-aj
   Return (x0,M) M=[m1..mn]. All solutions are x=x0+t*M

   Note: be carful with the overflow in the multiplication
*/

pair<ll, ll> linear_congruences(const vector<ll> &a, const vector<ll> &m)
{
   int n = a.size();
   ll u = a[0], v = m[0], p, q;
   for (int i = 1; i < n; ++i)
   {
        ll r = gcd(v, m[i], p, q);
        ll t = v;
        if ((a[i] - u) % r)
                return {-1, 0}; // no solution
        v = v / r * m[i];
        u = ((a[i] - u) / r * p * t + u) % v;
   }
   if (u < 0)
        u += v;
   return {u, v};
}
```

1.6.13. Mobius

```
//MOBIUS(X) is zero if X is not a square-free integer.
//MOBIUS(X) is 1 if X is a square-free integer and the
//number of X's prime divisors is even.
//MOBIUS(X) is -1 if X is a square-free integer and the
//number of X's prime divisors is odd.
int mobiusMu (Int n) {
   int ans = 1 ;
   for (Int d = 2 ; d <= n; ++d){
        if (n % (d * d) == 0 )
                return 0;
        else if (n % d == 0 )
                n /= d, ans *= -1;
   }
   return ans;
}
for(int i=2; i<=MAXA; i++)
   if(!P[i]){
        for(int j=i; j<=MAXA; j+=i)
                if(!P[j])
                        P[j]=i;
        }
 MOBIUS[1]=1;
    for(int i=2; i<=MAXA; i++)
        if(P[i]==P[i/P[i]])
            MOBIUS[i]=0;
        else
            MOBIUS[i]=-1 * MOBIUS[i/P[i]];
```

1.6.14. Modular Equations (ax = b(n))

```
/* Modular Linear Equation Solver O(log(n))
 * Given a, b and n, solves the equation ax = b(n)
 * for x. Returns the vector of solutions, all smaller
 * than n and sorted in increasing order. */
vector< int > msolve( int a, int b, int n ){
    if( n < 0 ) n = -n;
   int d, x, y;
    d = extended_euclid( a, n, x, y );
    vector< int > r;
    if( b % d ) return r;
    int x0 = ( b / d * x ) % n;
    if( x0 < 0 ) x0 += n;
   x0 = x0 % (n / d);
    for( int i = 0; i < d; i++ )
        r.push_back( ( x0 + i * n / d ) % n );
    return r;
}
```

1.6.15. Parametric Self-Dual Simplex method O(n+m)

```
/* - Solve a canonical LP:
                        min. c x
            s.t. A x <= b
                        x >= 0      */
const double eps = 1e-9, oo = numeric_limits<double>::infinity();
typedef vector<double> vec;
typedef vector<vec> mat;
double simplexMethodPD(mat &A, vec &b, vec &c){
   int n = c.size(), m = b.size();
   mat T(m + 1, vec(n + m + 1));
   vector<int> base(n + m), row(m);
   for(int j = 0; j < m; ++j){
            for (int i = 0; i < n; ++i)
                    T[j][i] = A[j][i];
            T[j][n + j] = 1;
            base[row[j] = n + j] = 1;
            T[j][n + m] = b[j];
   }
   for (int i = 0; i < n; ++i) T[m][i] = c[i];
```

```
    while (1){
            int p = 0, q = 0;
            for (int i = 0; i < n + m; ++i)
                    if (T[m][i] <= T[m][p]) p = i;
            for (int j = 0; j < m; ++j)
                    if (T[j][n + m] <= T[q][n + m]) q = j;
            double t = min(T[m][p], T[q][n + m]);
            if (t >= -eps)  {
                    vec x(n);
                    for (int i = 0; i < m; ++i)
                            if (row[i] < n) x[row[i]] = T[i][n + m];
                    // x is the solution
                    return -T[m][n + m]; // optimal
            }
            if (t < T[q][n + m]){
                    // tight on c -> primal update
                    for (int j = 0; j < m; ++j)
                            if (T[j][p] >= eps)
                                    if (T[j][p] * (T[q][n + m] - t) >=
                                            T[q][p] * (T[j][n + m] - t))
                                            q = j;

                    if (T[q][p] <= eps)
                            return oo; // primal infeasible
            }else{
                    // tight on b -> dual update
                    for (int i = 0; i < n + m + 1; ++i)
                            T[q][i] = -T[q][i];
                    for (int i = 0; i < n + m; ++i)
                            if (T[q][i] >= eps)
                                    if (T[q][i] * (T[m][p] - t) >=
                                            T[q][p] * (T[m][i] - t))
                                            p = i;
                    if (T[q][p] <= eps)
                            return -oo; // dual infeasible
            }
            for (int i = 0; i < m + n + 1; ++i)
                    if (i != p) T[q][i] /= T[q][p];
            T[q][p] = 1; // pivot(q, p)
            base[p] = 1;
            base[row[q]] = 0;
            row[q] = p;
            for (int j = 0; j < m + 1; ++j)
                    if (j != q){
                            double alpha = T[j][p];
                            for (int i = 0; i < n + m + 1; ++i)
                                    T[j][i] -= T[q][i] * alpha;
                    }
    }
    return oo;
}
```

1.6.16. Partition

```
ll partition(ll n){
 vector<ll> dp(n+1); dp[ 0] = 1;
 for (int i = 1; i <= n; i++)
 for (int j=1, r=1; i-(3*j*j-j)/2 >= 0; j++, r*=-1){
 dp[i] += dp[i-(3*j*j-j)/2]*r;
 if (i - (3*j*j+j)/2>=0)
 dp[i] += dp[i-(3*j*j+j)/2]*r;
 }
 return dp[n];
}
```

1.6.17. Phi

```
// computes the number of coprimes of p^k, being p prime
//int phi(int p, int k) { return pow(p, k) - pow(p, k-1); } // phi(p^k)
int phi(int p, int pk) { return pk - (pk/p); } // phi(p^k), where pk=p^k
// computes the number of coprimes of n
// phi(n) = (p_1 - 1) * p_1 ^ (k_1 - 1) * (p_2 - 1) * p_2 ^ (k_2-1)
int phi(int n){
    int coprimes = (n != 1); // phi(1) = 0
    for (int i = 2; i*i <= n; i++)
        if (n%i == 0){
            int pk = 1;
            while (n%i == 0)
                n /= i, pk *= i;
            coprimes *= phi(i, pk);
        }
    if (n > 1) coprimes *= phi(n, n); // n is prime
    return coprimes;
}

```

1.6.18. Pholard Rho(sirve)

```
typedef unsigned long long ull;
typedef long long ll;

bool Miller_Rabin(ll n){
   int a[5] = { 2, 3, 7, 61, 24251 };
   if (n == 2) return true;
   if (n == 1 || (n & 1) == 0) return false;
   ll b = n - 1;
   for (int i = 0; i < 5; i++)     {
           if (a[i] >= n) break;
           while ((b & 1) == 0) b >>= 1;
           ll t = exp_mod(a[i], b, n);
           while (b != n - 1 && t != 1 && t != n - 1){
                   t = mul_mod(t, t, n);
                   b <<= 1;
           }
           if (t == n - 1 || (b & 1)) continue;
           else return false;
   }
   return true;
}
ll factor[100];
ll nfactor, minfactor;
ll gcd(ll a, ll b){
   return b ? gcd(b, a%b) : a;
```

```
}
void Factor(ll n);
void pollard_rho(ll n){
    if (n <= 1) return;
    if (Miller_Rabin(n)){
            factor[nfactor++] = n;
            if (n < minfactor) minfactor = n;
            return;
    }
    ll x = 2 % n, y = x, k = 2, i = 1;
    ll d = 1;
    while (true){
            i++;
            x = (mul_mod(x, x, n) + 1) % n;
            d = gcd((y - x + n) % n, n);
            if (d > 1 && d < n)
            {
                    pollard_rho(d);
                    pollard_rho(n / d);
                    return;
            }
            if (y == x){
                    Factor(n);
                    return;
            }
            if (i == k){
                    y = x;
                    k <<= 1;
            }
    }
}

void Factor(ll n){
    ll d = 2;
    while (n % d != 0 && d * d <= n) d++;
    pollard_rho(d);
    pollard_rho(n / d);
}
// return true if n is prime
// nfactor is the size
bool factorize(ll n){
    nfactor = 0;
    minfactor = (1LL << 63);
    pollard_rho(n);
    if (nfactor == 1 && factor[0] == n){
            return true;
    }
    sort(factor, factor + nfactor);
    return false;
}

    ll n;
    factorize(n);

```

1.6.19. Shanks' Algorithm O(sqrt(N)) (a^x = b(mod m))

```
//return x such that a^x = b(mod m)
int solve ( int a, int b, int m ){
    int n = (int)sqrt( m + .0 )+1, an = 1;
    for ( int i = 0; i < n; i++ )
        an = (an * a)%m;
    map<int, int>vals;
    for ( int i = 1, cur = an; i <= n; ++ i ){
        if ( ! vals. count ( cur ) )
            vals [ cur ] = i ;
        cur = (cur * an)%m;
    }
    for ( int i = 0, cur = b; i <= n; ++ i ){
        if ( vals. count ( cur ) ){
            int ans = vals [ cur ] * n - i ;
            if ( ans < m )return ans;
        }
        cur = (cur * a)%m;
    }
    return -1;
}
```

1.6.20. Simpson Rule

```
// Error = O( (delta x)^4 )
 const int ITR = 1e4; //must be an even number
 double Simpson(double a, double b, double f(double) ){
 double s = f(a) + f(b), h = (b - a) / ITR;
 for (int i = 1; i < ITR; ++i) {
  double x = a + h * i;
  s += f(x)*( i&1 ? 4 : 2);
 }
 return s * h/3;
}
```

1.6.21. SQUARE_FREE.CPP

```
//recurse(1,1,m) calcula la cantidad de enteros menores o
//iguales que m que tienen un primo cuadrado como factor
   long long recurse(int idx, long long soFar,long long k){
      long long ret = 0;
      if(primesSquared[idx]*soFar > k)return 0;
      ret += (long long)(k/primesSquared[idx]/soFar);
      ret += recurse(idx+1,soFar,k);
      ret -= recurse(idx+1,soFar*primesSquared[idx],k);
      return ret;
   }
//si se quiere buscar el nth square free basta con buscar un m
//tal que m-recurse(1,1,m)==n y se puede hacer con busqueda binaria
```

1.7. String

1.7.1. Aho Corasick

```
int tree[MAXN][26], fail[MAXN];
int termina[MAXN], size = 1;
void addWord( string pal ){
    int p = 0;
    for(char c : pal){
```

```
        if( !tree[p][c-'a'] )
            tree[p][c-'a'] = size++;
        p = tree[p][c-'a'];
    }
    //termina[p].push_back( pal_id );
    termina[p] = pal.size();
}
void buildersuffix(){
    queue<int> Q;
    for(int i = 0; i < 26; i++)
        if( tree[0][i] ) Q.push(tree[0][i]);
    while( !Q.empty() ){
        int u, v = Q.front(); Q.pop();
           //for( auto i : termina[fail[v]] )
           //      termina[v].push_back( i );
        termina[v] = max(termina[v], termina[fail[v]]);
        for( int i = 0; i < 26; i++ )
            if(u = tree[v][i]){
                fail[u] = tree[fail[v]][i];
                Q.push( u );
            }else
                tree[v][i] = tree[fail[v]][i];
    }
}
```

1.7.2. Lyndon Decomposition O(N)

```
/*s = w1w2w3..wk, w1 >= w2 >=...>= wk.
  > Menor RotaciÃ³n LexicogrÃ¡fica:Es el mayor valor
  de i, tal que i < n, en la descomposicion de lyndon
  de la cadena s+s, n = |s|  */
void lyndon( string s ){
    int n = (int)s.length(), i = 0;
    while( i < n ){
        int j = i+1, k = i;
        while( j < n && s[k] <= s[j] ){
            if( s[k] < s[j] ) k = i;
            else k ++;
            j ++;
        }
        while( i <= k ){
            cout << s.substr( i, j-k )<<endl;
            i += j-k;
}    }    }
```

1.7.3. Manacher

```
/*rad[i] = If i is odd, it's the largest even palindrome centered at
position i.
Otherwise, it's the size of the largest odd palindrome centered at position
i.
*/
char s[maxn]; int rad[2 * maxn], n;
void build_rad () //WARNING: n = 2 * strlen(s) {
    int i, j, k;
   for ( i = 0, j = 0; i < 2 * n - 1; i += k ) {
    while ( i - j >= 0 && i + j + 1 < 2 * n &&
            s[ ( i - j ) / 2 ] == s[ ( i + j + 1 ) / 2 ] )
              j++;
        rad[i] = j;
    for ( k = 1; k <= rad[i] && rad[ i - k ] != rad[i] - k; k++ )
      rad[ i + k ] = min( rad[ i - k ], rad[i] - k );
        j = max( j - k, 0 );
  }
}
bool is_palindrome (int b, int e) {
   return b >= 0 && e < n &&  rad[b + e] >= e - b + 1;
}
```

1.7.4. Palindrome Tree

```
#define maxn  1000
struct Node{
    int start, end, length, insertEdg[26], suffixEdg;
};
Node tree[maxn], root1, root2;
int currNode, ptr;
string s;
void insert(int idx){
    int tmp = currNode;
    while (true){
        int curLength = tree[tmp].length;
        if (idx - curLength >= 1 and s[idx] == s[idx-curLength-1])
            break;
        tmp = tree[tmp].suffixEdg;
    }
    if(tree[tmp].insertEdg[s[idx]-'a'] != 0){
        currNode = tree[tmp].insertEdg[s[idx]-'a'];
        return;
    }
    ptr++;
    tree[tmp].insertEdg[s[idx]-'a'] = ptr;
    tree[ptr].length = tree[tmp].length + 2;
    tree[ptr].end = idx;
    tree[ptr].start = idx - tree[ptr].length + 1;
    tmp = tree[tmp].suffixEdg;
    currNode = ptr;
    if (tree[currNode].length == 1)    {
        tree[currNode].suffixEdg = 2;
        return;
    }
    while (true)    {
        int curLength = tree[tmp].length;
        if (idx-curLength >= 1 and s[idx] == s[idx-curLength-1])
            break;
        tmp = tree[tmp].suffixEdg;
    }
   tree[currNode].suffixEdg = tree[tmp].insertEdg[s[idx]-'a'];
   //do things with the tree...
}

void init_tree(){
   root1.length = -1;
```

```
    root1.suffixEdg = 1;
    root2.length = 0;
    root2.suffixEdg = 1;
    tree[1] = root1;
    tree[2] = root2;
    ptr = 2;
    currNode = 1;
}
/*
   init_tree();
   s = "abcbab";
    int l = s.length();
    for (int i=0; i<l; i++)
        insert(i);
    "All distinct palindromic substring"
    for (int i=3; i<=ptr; i++)
   palindrome en el nodo i [tree[i].start , tree[i].end]
*/
```

1.7.5. suffix automata O(N)

```
// Construct:
// Automaton sa; for(char c : s) sa.extend(c);
// 1. Number of distinct substr O( N ):
//    - Find number of different paths --> DFS on SA
//    - f[u] = 1 + sum( f[v] for v in s[u].next
// 2. Number of occurrences of a substr O( N ):
//    - Initially, in extend: s[cur].cnt = 1; s[clone].cnt = 0;
//    - for(it : reverse order)
//        p = nodes[it->second].link;
//        nodes[p].cnt += nodes[it->second].cnt
// 3. Find total length of different substrings O( N ):
//    - We have f[u] = number of strings starting from node u
//    - ans[u] = sum(ans[v] + d[v] for v in next[u])
// 4. Lexicographically k-th substring O(N)
//    - Based on number of different substring
// 5. Find first occurrence O(N)
//    - firstpos[cur] = len[cur] - 1, firstpos[clone] = firstpos[q]
// 6. Longest common substring of two strings s, t O(N).
struct state {
    int len, link;
    int fpos;///
    map<char,int>next;
    state( ){
        len = 0, link = -1, fpos = 0;
        next.clear();
    }
};
const int MAXLEN = 100002;
state st[MAXLEN*2];
int sz, last;
set<pair<int,int>> base ;///
int cnt[MAXLEN*2];///
void sa_init() {
    sz = last = 0;
    st[0] = state();
   cnt[0] = 0;
    sz++;
    base.clear();
}
void sa_extend (char c) {
    int cur = sz++;
    st[cur] = state();
    st[cur].len = st[last].len + 1;
    st[cur].fpos = st[cur].len - 1;///
    cnt[cur] = 1 ; ///
    base.insert(make_pair(st[cur].len, cur));///
    int p;
    for (p=last; p!=-1 && !st[p].next.count(c); p=st[p].link)
        st[p].next[c] = cur;
    if (p == -1)
        st[cur].link = 0;
    else {
        int q = st[p].next[c];
        if (st[p].len + 1 == st[q].len)
            st[cur].link = q;
        else {
            int clone = sz++;
            st[clone] = state();
            st[clone].len = st[p].len + 1;
            st[clone].next = st[q].next;
            st[clone].link = st[q].link;
            st[clone].fpos = st[q].fpos;///
            cnt[clone]=0;///
            base.insert(make_pair(st[clone].len,clone)); ///
            for (; p!=-1 && st[p].next[c]==q; p=st[p].link)
                st[p].next[c] = clone;
            st[q].link = st[cur].link = clone;
        }
    }
    last = cur;
}
//6. Longest common substring of two strings s, t.
string lcs (string s, string t) {
    sa_init();
    for (int i=0; i<(int)s.length(); i++)
        sa_extend (s[i]);
    int v = 0,  l = 0, best = 0,  bestpos = 0;
    for (int i=0; i<(int)t.length(); i++) {
        while (v && !st[v].next.count(t[i])) {
            v = st[v].link;
            l = st[v].len;
        }
        if (st[v].next.count(t[i])) {
            v = st[v].next[t[i]];
            l++;
        }
        if (l > best) best = l,  bestpos = i;
    }
    return t.substr (bestpos-best+1, best);
}
```

1.7.6. SUFFUX_ARRAY(N log N)

```
char s[MAX]; int rank[MAX], LCP [MAX];
int SA[MAX],wa[MAX], wb[MAX], we[MAX], wv[MAX],S[MAX],A[MAX];
void Sufix_Array(char *cad,int *SA,int N){
   N++;
   int i, j, p, *x = wa, *y = wb, range = 256;
   memset(we, 0, range * sizeof(int));
   for (i = 0; i < N; i++)
           we[ x[i] = cad[i] ]++;
   for (i = 1; i < range; i++) we[i] += we[ i-1 ];
   for (i = N - 1; i >= 0; i--)
           SA[ --we[ x[i] ] ] = i;
   for (j = p = 1; p < N; j <<= 1, range = p){
           for (p = 0, i = N - j; i < N; y[p++] = i , i++) ;
           for (i = 0; i < N; i++)
                   if (SA[i] >= j) y[p++] = SA[i] - j;
           for (i = 0; i < N; i++)
                   wv[i] = x[ y[i] ];
           memset(we, 0, range * sizeof(int));
           for (i = 0; i< N; i++)
                   we[ wv[i] ]++;
           for (i = 1; i < range; i++) we[i] += we[i-1];
           for (i = N-1; i >= 0; i--) SA[--we[wv[i]]] = y[i];
           swap(x, y); x[SA[0]] = 0;
           for (p = i = 1; i < N; i++)
                   if(y[SA[i]] == y[SA[i-1]] && y[SA[i]+j] == y[SA[i-1]+j])
                           x[SA[i]] = p - 1; else x[SA[i]] = p++;
   }
   N--;
}
void FindLCP(char *cad, int *SA, int N){
   int i, j, k;
   for (i = 1; i <= N; i++)
           rank[ SA[i] ] = i;
   for (k = i = 0; i < N; LCP [rank[i++]] = k)
           for (k ? k-- : 0,j = SA[rank[i]-1]; cad[i + k] == cad[j + k];
                           k++);
}
```

1.7.7. Tandems O(Nlog(N))

```
void output_tandem (const string & s, int shift,
           bool left, int cntr, int l, int l1, int l2){
   int pos;
   if (left) pos = cntr-l1;
   else pos = cntr-l1-l2-l1+1;
   cout << "[" << shift + pos << ".."; // ini
   cout << shift + pos+2*l-1 << "] = "; // fin
   cout << s.substr (pos, 2*l) << endl;
}
void output_tandems (const string & s, int shift,
           bool left, int cntr, int l, int k1, int k2){
   for (int l1=1; l1<=l; l1++) {
           if (left && l1 == l)  break;
           if (l1 <= k1 && l-l1 <= k2)
                   output_tandem(s,shift,left,cntr, l, l1, l-l1);
   }
}
inline int get_z (const vector<int> & z, int i) {
   return 0<=i && i<(int)z.size() ? z[i] : 0;
}
void find_tandems (string s, int shift = 0) {
   int n = (int) s.length();
   if (n == 1)  return;
   int nu = n/2,  nv = n-nu;
   string u = s.substr (0, nu),
           v = s.substr (nu);
   string ru = string (u.rbegin(), u.rend()),
           rv = string (v.rbegin(), v.rend());
   find_tandems (u, shift);
   find_tandems (v, shift + nu);
   vector<int> z1 = z_function (ru),
           z2 = z_function (v + '#' + u),
           z3 = z_function (ru + '#' + rv),
           z4 = z_function (v);
   for (int cntr=0; cntr<n; cntr++) {
           int l, k1, k2;
           if (cntr < nu) {
                   l = nu - cntr;
                   k1 = get_z (z1, nu-cntr);
                   k2 = get_z (z2, nv+1+cntr);
           }else {
                   l = cntr - nu + 1;
                   k1 = get_z (z3, nu+1 + nv-1-(cntr-nu));
                   k2 = get_z (z4, (cntr-nu)+1);
           }
           if(k1 + k2 >= l) // longitud 2*l
              output_tandems(s,shift,cntr<nu,cntr,l,k1,k2);
   }
}
```

1.7.8. Z_AND_PREFIX_FUNCTION

```
string s;
int z[100005];

    cin >> s;
    int n =  ( int ) s.length ( ) ;
    vector < int > pi ( n ) ;
    for  ( int I = 1 ; I < n ;  ++ I ){
        int j = pi [ I - 1 ] ;
        while  ( j > 0 && s [ I ]  != s [ j ] )
            j = pi [ j - 1 ] ;
        if  ( s [ I ]  == s [ j ] )   ++ j ;
        pi [ I ]  = j ;
    }
//cantidad de veces que aparece el prefijo de tamaño
//i en la cadena
    vector < int > ans ( n + 1 ) ;
    for  ( int I = 0 ; I < n ;  ++ I )
        ++ ans [ pi [ I ] ] ;
```

```
    for  ( int I = n - 1 ; I > 0 ;  -- I )
        ans [ pi [ I - 1 ] ]  += ans [ I ] ;

//Given a string S of length n,
//the Z Algorithm produces an array Z
//where Z[i] is the length of the longest
//substring starting from S[i] which is also a prefix of S
    int L = 0, R = 0;
    for (int i = 1; i < n; i++){
        if (i > R){
            L = R = i;
            while (R < n && s[R-L] == s[R]) R++;
            z[i] = R-L;
            R--;
        }
        else{
            int k = i-L;
            if (z[k] < R-i+1) z[i] = z[k];
            else{
                L = i;
                while (R < n && s[R-L] == s[R]) R++;
                z[i] = R-L;
                R--;
            }
        }
    }
```